CS 361L

Data Structures and Algorithms

Harshita Sahni

Haley Gray, Reuben Johnson, Ivan Barragan

The Department of Computer Science, University of New Mexico

Project 1

Backup of project 1: https://github.com/haleyg2/Project1/tree/main

Introduction:

In this project, we implemented and tested two different algorithm techniques. A 3-way merge sort and a Bloom filter for password breach checking. The goal of this project was to implement a 3-way merge sort and develop hashing techniques. For the sorting portion, we focused on implementing a merge sort variant that divides an array into three different subarrays instead of two, then it recursively sorts and merges them back together. For the hashing part of the project, we built a Bloom filter that can test whether a password hash may have appeared in a breach while using much less memory than a standard set or list.

The main purpose of the first part was to compare running time with actual measured performance. We were asked to test timing on input data of size: 2^{20} through 2^{30}, excluding the I/O time, and also asked for comparisons between integer inputs and double precision floating point inputs. It also asks for discussion of how asymptotic behavior matches the observed results and what input size can be handled before memory becomes a problem.

The second part of the project focused on designing a password breach checker using a Bloom filter. The Bloom filter stores information compactly using a shared bit array and multiple hash functions. This makes it possible for queries to process very quickly while using less memory than sorting all hashes directly. The Bloom filter allows false positives, so it must analyze how the false positives rate compares to the target and how the choices of m and k affect the performance.

Preliminaries and Methods:

3-Way Merge Sort:

Merge sort is a divide and conquer sorting algorithm. Normally a standard merge sort divides the array into two halves, recursively sorts each half, and merges them. In this project, we implemented a 3-way version, where the input array is divided into three different parts. Each part is sorted recursively, and then the three different parts are merged back into one sorted array. The expected asymptotic running time remains O(n log n), even though the recursion tree and merge behavior are different from that of a normal 2-way merge sort. The project requested benchmarking this implementation on large powers of two input sizes and comparing performance on integers and doubles.

Our implementation was written in Python and optimized using Numba with @numba.njit decorator to reduce Python overhead during the recursive calls and merging. We also used NumPy arrays for contiguous storage and better performance during array manipulation. Our

version switches to insertion sort for small subarrays, since insertion sort is faster than merge sort on tiny inputs due to lower overhead.

The main sorting function is mergeSort(arr, start, end). It first computes the size of the current subarray. If the subarray size is less than 16, the algorithm switches to insertion sort. Otherwise, it recursively divides the array into three different sections and sorts each section before merging them.

We originally used python lists after generating the random numbers but after some painful discovery, we noticed python lists take up 28 bytes per element; Consequently, we just kept it as a numpy list which was only using 8 bytes per element. We would have needed at least 60GB of RAM to merge on the top level of a list of size 2^{30}. With sticking to a numpy list, we only needed ~17GB. After this change & using Numba to speed things up, we were able to run the 3 way mergesort on all sizes 2^{20} to 2^{30} in a short amount of time with an older laptop from ~2020. Haley's laptop had 48GB of RAM which was able to handle all the sizes with no problems after the change.

Each mergesort run was timed in testRunner.py after a list of some size was generated and was stopped after sorting finished. testRunner.py is to be run two times; Once for Ints and another for Doubles. The line that generates the other was commented out on it for each. The results were put into a json file and were manually moved into the mergeSortPlot.py file which then displayed the results on a plot.

The code computes:

- Mid = (start + end) // 2
- First3rd = (start + mid) // 2
- Three4ths = (mid + end) // 2

Then it recursively sorts:

- arr[start : first3rd + 1]
- arr[first3rd + 1 : three4ths + 1]
- arr[three4ths + 1 : end + 1]

After the recursive calls, the function merge(arr, start, first3rd, three4ths, end) combines the three sorted regions back into the original array.

The merge function that we implemented creates three temporary arrays: L for the left portion, M for the middle portion, and R for the right portion. It copies the data from the original array into

these temporary arrays, then it repeatedly compares the front elements of the three arrays and writes the smallest one back into the original array. When one temporary array is exhausted, it continues to merge the remaining two, lastly, it copies over any elements that are left.

For optimization, a hybrid approach was used by switching to insertion sort for subarrays that are smaller than 16 elements. This allowed us to reduce recursive overhead, and it improves runtime. To evaluate performance, the sort should be run on input sizes 2^{20} through 2^{30}, which was required by the project. Input/ output time should be excluded, which means that the arrays should be generated or loaded before starting the timer. Separate timing experiments should be performed for integer arrays and double precision floating point arrays.

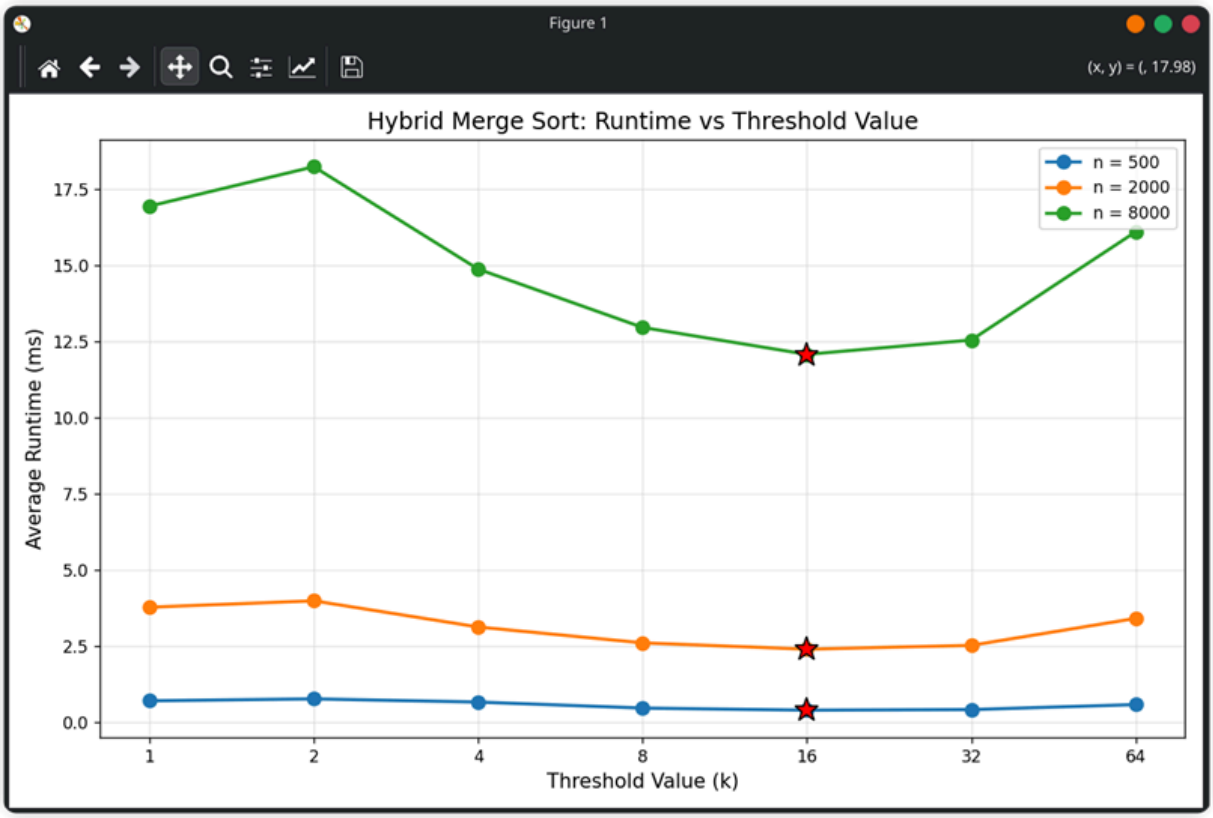
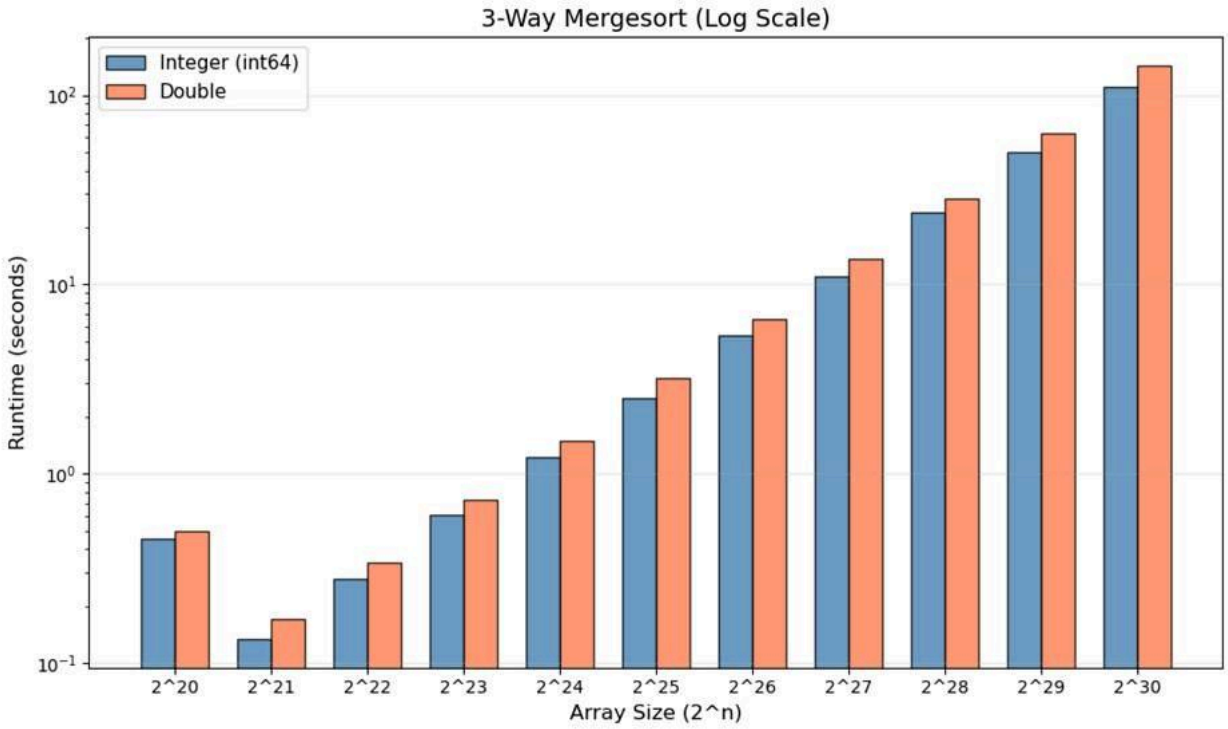


Figure 1
(x, y) = (, 17.98)
Hybrid Merge Sort: Runtime vs Threshold Value
n = 500
n = 2000
n = 8000
Average Runtime (ms)
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
1
2
4
8
16
32
64
Threshold Value (k)


3-Way Mergesort (Log Scale)
Integer (int64)
Double
Runtime (seconds)
10^2
10^1
10^0
10^{-1}
2^20
2^21
2^22
2^23
2^24
2^25
2^26
2^27
2^28
2^29
2^30
Array Size (2^n)

Bloom Filter:

The Bloom filter is a data structure which is used for testing in large datasets. The bloom filter provides fast insert and query operations while maintaining low memory usage. Additionally, the filter allows for false positives, which incorrectly identifies an element as present, while not allowing any false negatives.

The structure of the Bloom filter consists of a bit array of size m and k hash functions. When inserting an element, the element is passed through all of the k hash functions, which will then return an index in the bit array. The bits at those indices are set to 1. When querying, those same indices are checked. If any of the bits turn out to be 0, that element is not in the set. If all bits are 1 then the element is considered to be in the set, but it could be a false positive.

In our implementation, we loaded in 1,000,000 password hashes from a file and then we split them. The first 800,000 hashes were used as the positive set, and the remaining were used as the test for evaluating false positives.

The size of the bit array m and the number of hash functions k were calculated using the standard Bloom filter formulas based on the number of elements n = 800,000 and a target false positive rate of p = 0.01. The formulas help minimize the probability of collisions while keeping memory usage low. The bit array was implemented using a Python bytearray, which provides a compact and efficient way to store binary values. Although this is obviously one byte per position instead of one bit per position, the measured memory usage is larger than that of a fully packed bit-level implementation.

Instead of implementing independent hash functions, we used a double hashing technique. Two hash functions were implemented; the first is an FNV-1a hash and the second is a DJB2 hash. The FNV-1 hash uses XOR and multiplication with a prime constant to produce distributed hash values. The DJB2 hash uses repeated multiplication by 33 and addition of character values. We used the formula index = (h1 + i * h2) mod m to combine the hash functions. This allowed us to simulate multiple hash functions while keeping computation efficient.

The main insert function processes each hash in the training set by computing k indices and setting the corresponding bits in the bit array to 1. The query function uses the same indices and checks whether all corresponding bits are set. If any of the bits are zero then the function returns false. Otherwise, it just returns true.

To evaluate the performance of the Bloom filter, we ran two different tests. First, we queried all elements from the training set to verify if they are correct. Since we can not produce false negatives, all inserted elements should return true. Second, we queried all elements from the test

set and counted how many returned true. These were recorded as false positives, and the false positive rate was calculated as the ratio of false positives to the size of the test set.

We also measured performance using time.perf_counter() to track how long it took to query all test elements. We then were able to compute both the total query time and the average time. This showed us that the Bloom filter provides fast lookup times even for large datasets.

The Bloom filter achieves constant time insert and query operations, which is proportional to the number of hash functions k. It uses less memory than storing all the elements directly, and it maintains a controlled false positive rate close to the desired 1%. Making it a good solution for large membership testing problems where speed and memory efficiency are important.

False Positive Rate

So the total number of queries that returned true divided by the total number of queries is close to p but still off. Where $p = 0.01$ we got a rate of 0.0258, so the gap is 0.015 difference. We believed this was because the hash functions used didn't distribute values uniformly enough. Although before we had a false positive rate of 23% using a very simple hash function. This obviously was way too large of a gap so we had to optimize the hash function, fixing the uniform distribution problem but not fully.

Experiments	m value	k value	False Positive Rate %	Total Query Time for 1000 queries (seconds)	Average Query Time for 1000 Queries (seconds)	Bloom Filter Memory
Baseline	7,668,046	7	2.58%	0.008346	0.0000083460	7.31 MB
Larger m (m * 2)	15,336,092	13	1.095 %	0.0000101939	0.010194 second	14.63 MB
Smaller k (k-2)	7,668,046	5	2.1%	0.008828	0.0000088280 seconds	7.31 MB
Larger k (k + 2)	7,668,046	9	3.5%	0.009091	0.0000090913 seconds	7.31 MB

Effects of *k* and *m*

As you can see from the table above, as *k* increases the false positive rate grows similarly when you increase *k* the false positive rate grows as well. When you increase *m* the false positive rate actually lowers to a more precise measurement. Also you can observe that memory doubles for a larger *m*. So why can't you set both *m* and *k* to large values? This is because since *m* is the size of the bloom filter array it uses much more memory, so less space efficient. So it reduces false positives over the cost of memory. Since *k* is the number of hash functions if you make *k* larger, insertions are slower, queries are slower because we check/set more positions. So performance gets worse as *k* increases. And as you can see from the table, it also hurts the false positive rate when k grows.

Query Time

So we measured the time required for 1,000 keys and then computed the average time per query. Within the experiment 1,000 queries took about 0.008346 seconds total, which gives an average of about 0.000008346 seconds per query. So across the other experiments as well the general range was 8 to 10 microseconds per query. We believe this is fast enough for a real application, especially since a bloom filter is meant to provide quick member checks before expensive look ups.

Memory

So within the baseline experiment the filter used 7.31 MB of memory. When we doubled *m* the memory did so as well but decreased the false positive rate considerably (from 2.58% to 1.095%). This shows the expected tradeoff of using more memory improves accuracy. So storing the 800,000 hashes in a simple list or set of bit/byte array is much more memory efficient than every full hash string. So in conclusion the filter saves a significant amount of space while still supporting very fast queries.

Deletion

This is because you cannot simply flip the bits back to 0, those same bits may also have been set by other inserted hashes. So if you remove one hash by clearing all of its bits, you might also erase bits that another hash still needs. So that other hash could be queried later and incorrectly return false even though it was actually inserted. This ultimately would create false negatives.

Conclusion:

Once again, in this project, we implemented two different algorithm techniques with different goals. A 3-way merge sort for efficient sorting and a Bloom filter for memory efficient password breach checking. The merge sort portion allowed us to understand how a 3-way divide and conquer algorithm behaves in practice. Our benchmark results showed us that the implementation follows the O(n log n) trend.

The Bloom filter portion allowed us to understand how data structures can provide fast testing with low memory usage. Even though the Bloom filter allowed false positives, they remained useful because they allowed us to understand that any negative result is true. Our results demonstrated the expected tradeoff between memory, speed, and accuracy, and showed why Bloom filters are effective for applications such as password breach checkers.

Project 1 helped us better understand both algorithms and data structures, as well as analyzing different implementations.

Bibliography:

1. Spring 2026, CS361 Programming project: Sorting and Hashing, The University of New Mexico.

2. Have I Been Pwned, “Pwned Password.”

3. Course notes and lecture material from CS361

4. Three Way Merge Sort Provided on - GitHub: Project1/test3waymatrix at main · haleyg2/Project1

5. Initially our false positive rate was way too large to continue, so we ended up using conventional hash functions that are spreading out values uniformly enough to fix our false positive rate:

- https://www.ietf.org/archive/id/draft-eastlake-fnv-21.html The first hash function is from the FNV family of non-cryptographic hash functions. Specifically, the command FNV constant hash is a power-of-two number of bits (32, 64, 128, 256, 512, or 1024) and offset_basis and FNV_Prime depend on the size of the hash.
- https://theartincode.stanis.me/008-djb2/ This is the second hash function - This function/method is a "djb2 hash" written by Daniel J. Bernstein.

Contribution of Each Team Member:

Haley Gray implemented the 3-way merge sort code, including the recursive structure, merge procedure, and insertion sort optimization. Haley also contributed to generating the plots and building the report and worked on the Bloom filter, including hash testing, and performance evaluation. Reuben Johnson assisted with the 3-way merge sort, worked on the Bloom filter portion, including hashing, debugging, analyzing the results, and building the report. Ivan Barragan worked on the 3- way merge sort, assisted with the Bloom filter, and helped write and organize the report. In part 2 of the project Reuben downloaded the "pwnedpasswords" and parsed the first million lines for testing(phase 1). Haley implemented phase 2 and 3 while Ivan and Reuben worked on testing and optimizing the bloom filter. Reuben then wrote up the experimentation used in phase 5. We all worked together to write the report.